Q1. Collect business data and perform EDA

Aim: To perform Exploratory Data Analysis on business sales data and identify useful customer and sales trends.

Python Code

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("sales.csv")
print(df.head())
df.info()
print(df.describe())

print("Missing Values:")
print(df.isnull().sum())
print("Duplicate Rows:", df.duplicated().sum())

print(df["Product"].value_counts())
sns.countplot(data=df, x="Product")
plt.title("Product Distribution")
plt.xticks(rotation=45)
plt.show()

sns.countplot(data=df, x="Gender")
plt.title("Gender Distribution")
plt.show()

sns.countplot(data=df, x="City")
plt.title("City Distribution")
plt.xticks(rotation=45)
plt.show()

plt.hist(df["Price"], bins=10, edgecolor="black")
plt.title("Price Distribution")
plt.xlabel("Price")
plt.ylabel("Frequency")
plt.show()

sns.boxplot(y=df["Total"])
plt.title("Total Sales Box Plot")
plt.show()

sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="Blues")
plt.title("Correlation Heatmap")
plt.show()

print("Total Revenue:", df["Total"].sum())
print("Average Transaction:", df["Total"].mean())
print(df.groupby("Product")["Total"].sum())
print(df.groupby("City")["Total"].sum())
print(df.groupby("Gender")["Total"].mean())
```

Conclusion: EDA helps identify sales patterns, popular products, customer characteristics, and opportunities for business improvement.

Q2. Apply the Central Limit Theorem

Aim: To study the sampling distribution of sample means and compare it with the population mean.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

population = np.random.randint(100, 1000, 1000)
print("Population Mean:", np.mean(population))

sample_means = []
for i in range(1000):
    sample = np.random.choice(population, size=30, replace=True)
    sample_means.append(np.mean(sample))

print("Mean of Sample Means:", np.mean(sample_means))

plt.hist(sample_means, bins=30, edgecolor="black")
plt.title("Sampling Distribution of Sample Means")
plt.xlabel("Sample Mean")
plt.ylabel("Frequency")
plt.show()
```

Conclusion: Under suitable conditions, the distribution of sample means becomes approximately normal as sample size increases. Its mean tends to be close to the population mean.

Q3. Calculate descriptive statistics using Sales.csv

Aim: To calculate mean, median, mode, variance, standard deviation, minimum, maximum, and range.

Python Code

```
import pandas as pd

df = pd.read_csv("sales.csv")
sales = df["Total"]

print("Summary Statistics:")
print(sales.describe())
print("Mean:", sales.mean())
print("Median:", sales.median())
print("Mode:", sales.mode().tolist())
print("Variance:", sales.var())
print("Standard Deviation:", sales.std())
print("Minimum Sales:", sales.min())
print("Maximum Sales:", sales.max())
print("Range:", sales.max() - sales.min())
```

Conclusion: Descriptive statistics summarize the centre, spread, and range of transaction values.

Q4. Calculate a confidence interval for the population mean

Aim: To calculate a 95% confidence interval for the population mean using sample sales data.

Note: This code calculates a confidence interval for a mean. If your question requires a proportion too, an additional calculation is needed.

Python Code

```
import pandas as pd
from scipy import stats

df = pd.read_csv("sales.csv")
sales = df["Total"].dropna()

mean = sales.mean()
n = len(sales)
standard_error = stats.sem(sales)

lower, upper = stats.t.interval(
    0.95, df=n-1, loc=mean, scale=standard_error
)

print("Sample Mean:", mean)
print("Sample Size:", n)
print("Standard Error:", standard_error)
print("95% Confidence Interval:", lower, "to", upper)
```

Conclusion: The interval gives a range of plausible values for the population mean using a 95% confidence procedure.

Q5. Create a histogram and box plot

Aim: To visualize the distribution of total sales and identify possible outliers.

Python Code

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("sales.csv")
sales = df["Total"].dropna()

plt.figure(figsize=(8, 5))
plt.hist(sales, bins=10, edgecolor="black")
plt.title("Distribution of Total Sales")
plt.xlabel("Total Sales")
plt.ylabel("Frequency")
plt.show()
```

```
plt.figure(figsize=(6, 5))
sns.boxplot(y=sales)
plt.title("Box Plot of Total Sales")
plt.ylabel("Total Sales")
plt.show()
```

Conclusion: The histogram shows frequency and distribution shape. The box plot shows the median, spread, and potential outliers.

Q6. Apply bootstrapping to estimate a confidence interval

Aim: To estimate a 95% confidence interval for the median using bootstrap sampling.

Python Code

```
import pandas as pd
import numpy as np

df = pd.read_csv("sales.csv")
sales = df["Total"].dropna().values

bootstrap_medians = []
for i in range(1000):
    sample = np.random.choice(sales, size=len(sales), replace=True)
    bootstrap_medians.append(np.median(sample))

lower = np.percentile(bootstrap_medians, 2.5)
upper = np.percentile(bootstrap_medians, 97.5)

print("Original Median:", np.median(sales))
print("95% Confidence Interval:", lower, "to", upper)
```

Conclusion: Bootstrapping estimates uncertainty by repeatedly taking samples with replacement. The 2.5th and 97.5th percentiles give an approximate 95% confidence interval.

Q7. Perform correlation analysis

Aim: To determine the strength and direction of relationships between advertising expenditure and sales revenue.

Python Code

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("advertising_sales Pract3.csv")
print(df.head())
df.info()

cols = ["TV_Advertising", "Radio_Advertising",
        "Newspaper_Advertising", "Sales_Revenue"]
correlation = df[cols].corr()
print("Correlation Matrix:")
print(correlation)

plt.figure(figsize=(8, 6))
sns.heatmap(correlation, annot=True, cmap="coolwarm", linewidths=0.5)
plt.title("Correlation Matrix")
plt.show()

plt.scatter(df["TV_Advertising"], df["Sales_Revenue"])
plt.title("TV Advertising vs Sales Revenue")
plt.xlabel("TV Advertising")
plt.ylabel("Sales Revenue")
plt.show()
```

Conclusion: Correlation measures the strength and direction of a linear relationship. Correlation alone does not prove causation.

Q8. Perform hypothesis testing for a business problem

Aim: To test whether the population mean sales differs significantly from a specified benchmark.

Note: ■10,000 is an example benchmark, not verified from your guide. Replace it with the value in your actual question.

Python Code

```
import pandas as pd
from scipy.stats import ttest_1samp

df = pd.read_csv("sales.csv")
sales = df["Total"].dropna()

# H0: Population mean = 10000
# H1: Population mean is not equal to 10000
t_stat, p_value = ttest_1samp(sales, 10000)

print("T-statistic:", t_stat)
print("P-value:", p_value)

if p_value < 0.05:
    print("Reject the Null Hypothesis")
else:
    print("Fail to Reject the Null Hypothesis")
```

Conclusion: If p-value is less than 0.05, reject the null hypothesis; otherwise, fail to reject it.

Q9. Simulate a dice-rolling experiment

Aim: To simulate dice rolls and compare experimental probability with theoretical probability.

Python Code

```
import random

n = 10000
count = 0

for i in range(n):
    result = random.randint(1, 6)
    if result == 6:
        count += 1

experimental_probability = count / n
theoretical_probability = 1 / 6

print("Total Rolls:", n)
print("Number of Sixes:", count)
print("Experimental Probability:", experimental_probability)
print("Theoretical Probability:", theoretical_probability)
```

Conclusion: The theoretical probability of getting a six on a fair die is 1/6. Experimental probability tends to approach it over many trials.

Q10. Generate random numbers from three distributions

Aim: To generate normal, uniform, and exponential random numbers and study them using histograms.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

normal_data = np.random.normal(50, 10, 1000)
uniform_data = np.random.uniform(0, 100, 1000)
exponential_data = np.random.exponential(10, 1000)

plt.hist(normal_data, bins=30, edgecolor="black")
plt.title("Normal Distribution")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()

plt.hist(uniform_data, bins=30, edgecolor="black")
plt.title("Uniform Distribution")
plt.show()

plt.hist(exponential_data, bins=30, edgecolor="black")
plt.title("Exponential Distribution")
plt.show()

print("Normal Mean:", np.mean(normal_data))
print("Uniform Mean:", np.mean(uniform_data))
print("Exponential Mean:", np.mean(exponential_data))
```

Conclusion: The normal distribution is bell-shaped, the uniform distribution spreads values evenly, and the exponential distribution has many smaller values and fewer large values.

Q11. Develop a decision tree for a business decision

Aim: To develop a decision tree model that predicts a business decision from input data.

Note: This is a demonstration classification example; the exact decision-tree code from your guide was not provided.

Python Code

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

# Example data: demand level
X = [[1], [2], [3], [4], [5], [6]]
# 0 = Do not launch, 1 = Launch
y = [0, 0, 0, 1, 1, 1]

model = DecisionTreeClassifier(max_depth=3, random_state=0)
model.fit(X, y)

prediction = model.predict([[4]])
print("Predicted Decision:", prediction[0])

plt.figure(figsize=(8, 5))
plot_tree(model, feature_names=["Demand Level"],
          class_names=["Do Not Launch", "Launch"], filled=True)
plt.show()
```

Conclusion: A decision tree represents decisions through branches and outcomes. For uncertainty problems, include outcomes, probabilities, and payoffs.

Q12. Calculate expected value and select the optimal alternative

Aim: To calculate expected values for two business alternatives and select the option with the higher expected profit.

Note: The profits and probabilities are illustrative examples, not confirmed values from your professor's guide.

Python Code

```
# Possible profits for Option A
profit_A = [1000, 5000]
prob_A = [0.5, 0.5]

# Possible profits for Option B
profit_B = [2000, 3000]
prob_B = [0.5, 0.5]

ev_A = sum(p * v for p, v in zip(prob_A, profit_A))
ev_B = sum(p * v for p, v in zip(prob_B, profit_B))

print("Expected Value of Option A:", ev_A)
print("Expected Value of Option B:", ev_B)

if ev_A > ev_B:
    print("Choose Option A")
elif ev_B > ev_A:
    print("Choose Option B")
else:
    print("Both options have equal expected value")
```

Conclusion: Expected value is the sum of each possible outcome multiplied by its probability. Option A has the higher expected profit in this example.